through Java Web Start (JNLP). In this approach, the user will log in to the slave node, open a browser and open the slave page by pointing to the URL of the master system. It may look like the following URL: *http://masterserver:PORT/hudson/jnlpJars/slave.jar*

You are then presented with the JNLP launch icon. If you click on the icon, the Java Web Start kicks in and it launches a slave agent on your computer.

This mode is convenient when the master cannot initiate a connection to slaves, such as when it runs outside a firewall while the rest of the slaves are within the firewall. The disadvantage is, if the machine with a slave agent goes down, the master has no way of re-launching it on its own.

Launching a slave agent 'headlessly'

This launch mode uses a mechanism very similar to Java Web Start, except that it can run without using a GUI—making it convenient to execute as a daemon on UNIX systems. To do this, you should configure this slave to be a JNLP slave by downloading *slave.jar*, and then from the slave, run the following command:

```
java -jar slave.jar -jnlpUrl \
http://masterserver:PORT/computer/slave-
name/slave-agent.jnlp
```

The *slave.jar* file is downloaded from the above-mentioned URL. Make sure to replace *slave-name* with the name of the slave set-up in the master.

Tips: By default, Hudson runs on port 8080. It can be installed and managed without the need for super user privileges.

Launching a slave agent using your own scripts

If the above modes are not flexible, you can even write your own script to launch a slave agent. The script is placed in the master computer and Hudson runs this script whenever it should connect to the slave. The script may use a remote log-in program like SSH or RSH to establish a connection between the master and slave.

The script would execute the slave agent program like *java -jar slave.jar*. The STDIN and STDOUT for the script should be connected to the master. For example, the script that executes *ssh myslave java -jar ~/bin/slave.jar* from the master Web interface would satisfy this need. For precisely this reason, running this script manually from the command line does no good.

Launching a slave agent using this mode requires an additional set-up on the slave. The benefit is that when the connection goes bad, you can use the Hudson Web interface to re-establish the connection.

Note: You should update *slave.jar* every time the Hudson system is upgraded. Though, in practice, *slave.jar* is not changed frequently.

Launching slave agents on Windows machines as a service is out of the scope of this article.

Hopefully, the above discussion has intrigued you enough to make you configure a simple Hudson job that performs certain tasks that are specific to a build and a test. Once you start using it, you will definitely be interested in its scope to execute distributed builds. END

By: Bhuvaneswaran A and Nithya Pattabiraman

Bhuvaneswaran is part of the Production Engineering Group at CollabNet and contributes to Ubuntu and Subversion projects during his spare time. You can find more about him at http://livecipher.com. Nithya is part of the Quality Assurance team at CollabNet and you can reach her at nithya@collab.net.